by

Kashaf Fatima 22i-2415

Safi ur Rehman 22i-2534

Amna Noor 22i-1529

Software Reengineering

Assignment 1

Submitted to [Nigar Azhar Butt]

Date: 9/14/2025

Table of Contents

A. Group Report

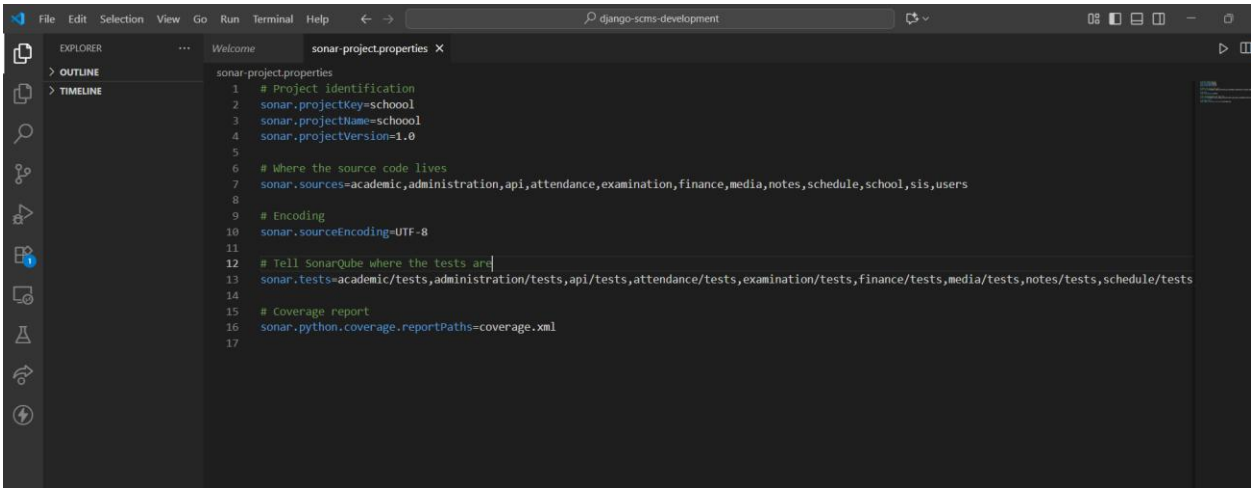
1. Project Overview

- ### Repository URL and commit hash

  Repository URL : https://github.com/mwinamijr/django-scms.git

  Commit hash : 6c3ca2cf0c59b19ba358b36313fbff0a70009f71

  Here is the properties file that we included in the project for sonarqube:



- ### System description and purpose

  For managing a school or college, this is an open source school management system. The API was created using Django and the Django-rest framework. For administration, admission, attendance, scheduling, and the results, it offers an API. Depending on their level of access, it also grants users varying permissions to utilize different apps.

  Key Features:

  1. Authentication and Role-Based Access Control

     Support authentication and permission control for:

     - Admins
     - Teachers
     - Accountants
     - Parents

2. Finance module
    - Manage Receipts
    - Track Payments
    - Generate Financial Reports

3. School Information System (SIS)
    - Tracks student records and their associated parent/guardian contacts
    - Manages class and academic year data
    - Required module for all other apps

4. Admissions
    - Manages student admission pipeline and levels
    - Tracks marketing channels and open house participation

5. Schedule Management
    - Enables creation and management of class periods with assigned teachers, subjects, classrooms, and terms.
    - Provides automatic timetable generation through a management command.

6. Examinations and Grading
    - Provides exam scheduling and status tracking (upcoming, ongoing, completed).
    - Manages student marks, GPA calculation, and validation against grading rules.
    - Supports flexible grade scales (letter or numeric) with consistency checks.

7. Digital Notes and Materials
    - Allows teachers to create assignments with questions, multiple-choice options, and correct answers.
    - Provides automatic grading for student submissions and stores results.
    - Supports structured learning materials through concepts, explanations, and topic-linked notes with images and examples.

8. Attendance Tracking
    - Records and manages daily attendance for teachers and students, including period-level tracking.
    - Supports detailed status codes (present, absent, late, half-day, excused) for accurate reporting.
    - Provides CRUD APIs for viewing, adding, updating, and deleting attendance records.

9. Academic module
    - Manages core school entities including students, parents, teachers, classes, dormitories, and subjects.
    - Automates enrollments, debt tracking, capacity management, and role-based access for users.
    - Supports academic history, medical records, messaging, and structured file/document storage.

10. Administration module
    - Manages academic years, terms, and school events with validation, filtering, and bulk upload via Excel templates.
    - Provides article and carousel image publishing features for school announcements and homepage content.
    - Tracks school info and access logs, ensuring accountability, structured scheduling, and communication.

- ## Approximate LOC and module count

  LOC : 6610

  Module Count :10

2. Inventory Analysis

o List of project assets (code, libraries, configurations, tests, documentation, UI)

&

o Classification table (e.g., Active / Deprecated / Reusable)

ID	Path	Filename	Asset Type	Classification
1	django-scms/	.gitignore	Configuration	Active
2	django-scms/.vscode/	settings.json	Configuration	Active
3	django-scms/academic/	__init__.py	Code	Active
4	django-scms/academic/	admin.py	Code	Active
5	django-scms/academic/	apps.py	Code	Active
6	django-scms/academic/management/commands/	__init__.py	Code	Active
7	django-scms/academic/m	update_student_debt.py	Code	Active

	anagement/commands/			
8	django-scms/academic/management/commands/	update_unpaid_salaries.py	Code	Active
9	django-scms/academic/migrations/	__init__.py	Code	Reusable
10	django-scms/academic/migrations/	0001_initial.py	Code (Migrations)	Reusable
11	django-scms/academic/migrations/	0002_initial.py	Code (Migrations)	Reusable
12	django-scms/academic/migrations/	0003_allocatedsubject_delete_subjectallocation.py	Code (Migrations)	Reusable
13	django-scms/academic/migrations/	0004_remove_classroom_grade_level_and_more.py	Code (Migrations)	Reusable
14	django-scms/academic/migrations/	0005_alter_subject_options_remove_topic_class_room_and_more.py	Code (Migrations)	Reusable
15	django-scms/academic/migrations/	0006_remove_studentclass_student_id_studentclass_student_and_more.py	Code (Migrations)	Reusable

16	django-scms/academic/migrations/	0007_alter_parent_options_alter_student_options_and_more.py	Code (Migrations)	Reusable
17	django-scms/academic/migrations/	0008_alter_subject_options_alter_teacher_options.py	Code (Migrations)	Reusable
18	django-scms/academic/migrations/	0009_alter_student_options_remove_teacher_isteacher.py	Code (Migrations)	Reusable
19	django-scms/academic/migrations/	0010_student_std_vii_number.py	Code (Migrations)	Reusable
20	django-scms/academic/migrations/	0011_rename_prem_number_student_prems_number.py	Code (Migrations)	Reusable
21	django-scms/academic/migrations/	0012_alter_student_parent_guardian.py	Code (Migrations)	Reusable
22	django-scms/academic/migrations/	0013_remove_student_debt.py	Code (Migrations)	Reusable
23	django-scms/academic/migrations/	0014_rename_studentclass_studentclassenrollment.py	Code (Migrations)	Reusable

24	django-scms/academic/migrations/	0015_alter_student_date_of_birth.py	Code (Migrations)	Reusable
25	django-scms/academic/	models.py	Code	Active
26	django-scms/academic/	serializers.py	Code	Active
27	django-scms/academic/	tests.py	Tests	Active
28	django-scms/academic/	validators.py	Code	Active
29	django-scms/academic/	views.py	Code	Active
30	django-scms/administration/	__init__.py	Code	Active
31	django-scms/administration/	admin.py	Code	Active
32	django-scms/administration/	apps.py	Code	Active
33	django-scms/administration/	common_objs.py	Code	Active

34	django-scms/administration/migrations/	__init__.py	Code	Reusable
35	django-scms/administration/migrations/	0001_initial.py	Code (Migrations)	Reusable
36	django-scms/administration/migrations/	0002_initial.py	Code (Migrations)	Reusable
37	django-scms/administration/migrations/	0003_alter_term_default_term_fee.py	Code (Migrations)	Reusable
38	django-scms/administration/migrations/	0004_schoolevent.py	Code (Migrations)	Reusable
39	django-scms/administration/migrations/	0005_remove_academicyear_graduation_date.py	Code (Migrations)	Reusable
40	django-scms/administration/	models.py	Code	Active
41	django-scms/administration/	permissions.py	Code	Active

42	django-scms/administration/	serializers.py	Code	Active
43	django-scms/administration/	tests.py	Tests	Active
44	django-scms/administration/	views.py	Code	Active
45	django-scms/api/	__init__.py	Code	Active
46	django-scms/api/academic/	urls.py	Code	Active
47	django-scms/api/	admin.py	Code	Active
48	django-scms/api/administration/	urls.py	Code	Active
49	django-scms/api/	apps.py	Code	Active
50	django-scms/api/assignments/	urls.py	Code	Active
51	django-scms/api/attendance/	urls.py	Code	Active

52	django-scms/api/blog/	urls.py	Code	Active
53	django-scms/api/	exceptions.py	Code	Active
54	django-scms/api/finance/	urls.py	Code	Active
55	django-scms/api/journals/	urls.py	Code	Active
56	django-scms/api/	middleware.py	Code	Active
57	django-scms/api/migrations/	__init__.py	Code	Reusable
58	django-scms/api/notes/	urls.py	Code	Active
59	django-scms/api/schedule/	urls.py	Code	Active
60	django-scms/api/	serializers.py	Code	Active
61	django-scms/api/sis/	urls.py	Code	Active
62	django-scms/api/users/	urls.py	Code	Active
63	django-scms/api/	views.py	Code	Active

64	django-scms/attendance/	**init**.py	Code	Active
65	django-scms/attendance/	admin.py	Code	Active
66	django-scms/attendance/	apps.py	Code	Active
67	django-scms/attendance/migrations/	**init**.py	Code (Migrations)	Reusable
68	django-scms/attendance/migrations/	0001_initial.py	Code (Migrations)	Reusable
69	django-scms/attendance/	models.py	Code	Active
70	django-scms/attendance/	serializers.py	Code	Active
71	django-scms/attendance/	tests.py	Tests	Active
72	django-scms/attendance/	views.py	Code	Active
73	django-scms/	db.sqlite3	Database	Active
74	django-scms/examination/	**init**.py	Code	Active

75	django-scms/examination/	admin.py	Code	Active
76	django-scms/examination/	apps.py	Code	Active
77	django-scms/examination/migrations/	**init**.py	Code (Migrations)	Reusable
78	django-scms/examination/migrations/	0001_initial.py	Code (Migrations)	Reusable
79	django-scms/examination/	models.py	Code	Active
80	django-scms/examination/	serializers.py	Code	Active
81	django-scms/examination/	tests.py	Tests	Active
82	django-scms/examination/	views.py	Code	Active
83	django-scms/finance/	**init**.py	Code	Active
84	django-scms/finance/	admin.py	Code	Active

85	django-scms/finance/	apps.py	Code	Active
86	django-scms/finance/migrations/	**init**.py	Code (Migrations)	Reusable
87	django-scms/finance/migrations/	0001_initial.py	Code (Migrations)	Reusable
88	django-scms/finance/migrations/	0002_initial.py	Code (Migrations)	Reusable
89	django-scms/finance/migrations/	0003_alter_payment_user.py	Code (Migrations)	Reusable
90	django-scms/finance/migrations/	0004_remove_payment_payment_no_remove_receipt_receipt_no_and_more.py	Code (Migrations)	Reusable
91	django-scms/finance/migrations/	0005_receipt_paid_through_alter_receipt_payer_debtrecord.py	Code (Migrations)	Reusable
92	django-scms/finance/migrations/	0006_receipt_payment_date.py	Code (Migrations)	Reusable
93	django-scms/finance/migrations/	0007_payment_paid_through_alter_receipt_paid_through.py	Code (Migrations)	Reusable

94	django-scms/finance/migrations/	0008_alter_debtrecord_options_debtrecord_amount_paid_and_more.py	Code (Migrations)	Reusable
95	django-scms/finance/migrations/	0009_receipt_term.py	Code (Migrations)	Reusable
96	django-scms/finance/	models.py	Code	Active
97	django-scms/finance/	serializers.py	Code	Active
98	django-scms/finance/	tests.py	Tests	Active
99	django-scms/finance/	views.py	Code	Active
100	django-scms/	manage.py	Code	Active
101	django-scms/media/articles/	Copy_of_HIC_logo_KnCOfnt.jpg	Assets (Media)	Active
102	django-scms/media/articles/	Copy_of_HIC_logo.jpg	Assets (Media)	Active
103	django-scms/media/articles/	Screenshot_from_2020-12-21_09-39-21.png	Assets (Media)	Active
104	django-scms/notes/	**init**.py	Code	Active

105	django-scms/notes/	admin.py	Code	Active
106	django-scms/notes/	apps.py	Code	Active
107	django-scms/notes/migrations/	**init**.py	Code (Migrations)	Reusable
108	django-scms/notes/migrations/	0001_initial.py	Code (Migrations)	Reusable
109	django-scms/notes/migrations/	0002_initial.py	Code (Migrations)	Reusable
110	django-scms/notes/	models.py	Code	Active
111	django-scms/notes/	serializers.py	Code	Active
112	django-scms/notes/	tests.py	Tests	Active
113	django-scms/notes/	views.py	Code	Active
114	django-scms/	Procfile	Configuration	Active
115	django-scms/	README.md	Documentation	Active
116	django-scms/	requirements.txt	Libraries	Active
117	django-scms/schedule/	**init**.py	Code	Active

118	django-scms/schedule/	admin.py	Code	Active
119	django-scms/schedule/	apps.py	Code	Active
120	django-scms/schedule/management/commands/	generate_timetable.py	Code	Active
121	django-scms/schedule/migrations/	**init**.py	Code (Migrations)	Reusable
122	django-scms/schedule/migrations/	0001_initial.py	Code (Migrations)	Reusable
123	django-scms/schedule/	models.py	Code	Active
124	django-scms/schedule/	serializers.py	Code	Active
125	django-scms/schedule/	tests.py	Tests	Active
126	django-scms/schedule/	views.py	Code	Active
127	django-scms/school/	**init**.py	Code	Active

128	django-scms/school/	asgi.py	Code / Configuration	Active
129	django-scms/school/	settings.py	Configuration	Active
130	django-scms/school/	urls.py	Code	Active
131	django-scms/school/	wsgi.py	Code	Active
132	django-scms/sis/	**init**.py	Code	Active
133	django-scms/sis/	admin.py	Code	Active
134	django-scms/sis/	apps.py	Code	Active
135	django-scms/sis/migrations/	**init**.py	Code (Migrations)	Reusable
136	django-scms/sis/migrations/	0001_initial.py	Code (Migrations)	Reusable
137	django-scms/sis/	models.py	Code	Active
138	django-scms/sis/	serializers.py	Code	Active
139	django-scms/sis/	tests.py	Tests	Active
140	django-scms/sis/	views.py	Code	Active
141	django-scms/static/images/articles/	Copy_of_HIC_logo.jpg	Assets (UI/Media)	Active

142	django-scms/static/images/carousel/	20200918_100730.jpg	Assets (UI/Media)	Active
143	django-scms/static/images/carousel/	20200921_093334.jpg	Assets (UI/Media)	Active
144	django-scms/static/images/carousel/	20200921_093729.jpg	Assets (UI/Media)	Active
145	django-scms/static/images/nursery/	20201128_091608.jpg	Assets (UI/Media)	Active
146	django-scms/static/images/nursery/	20201128_091634.jpg	Assets (UI/Media)	Active
147	django-scms/static/images/primary/	20201128_092423.jpg	Assets (UI/Media)	Active
148	django-scms/static/images/primary/	20201128_092541.jpg	Assets (UI/Media)	Active
149	django-scms/templates/	index.html	UI	Active
150	django-scms/users/	**init**.py	Code	Active
151	django-scms/users/	admin.py	Code	Active

152	django-scms/users/	apps.py	Code	Active
153	django-scms/users/	forms.py	Code	Active
154	django-scms/users/	managers.py	Code	Active
155	django-scms/users/migrations/	**init**.py	Code (Migrations)	Reusable
156	django-scms/users/migrations/	0001_initial.py	Code (Migrations)	Reusable
157	django-scms/users/migrations/	0002_alter_customuser_options_and_more.py	Code (Migrations)	Reusable
158	django-scms/users/migrations/	0003_customuser_phone_number.py	Code (Migrations)	Reusable
159	django-scms/users/	models.py	Code	Active
160	django-scms/users/	serializers.py	Code	Active
161	django-scms/users/	tests.py	Tests	Active
162	django-scms/users/	views.py	Code	Active

o Quality Metrics

1. Cyclomatic complexity

The total cyclomatic complexity of 593 across the entire codebase indicates significant accumulated complexity that requires systematic refactoring attention. While this represents the sum across all modules, individual functions should be evaluated against the ≤10 threshold for optimal maintainability.

Overall cyclomatic complexity = 593
Here is the root folder cyclomatic complexity:

academic = 158

administration = 44

api = 16

attendance = 34

examination = 35

finance = 110

notes = 22

schedule = 15

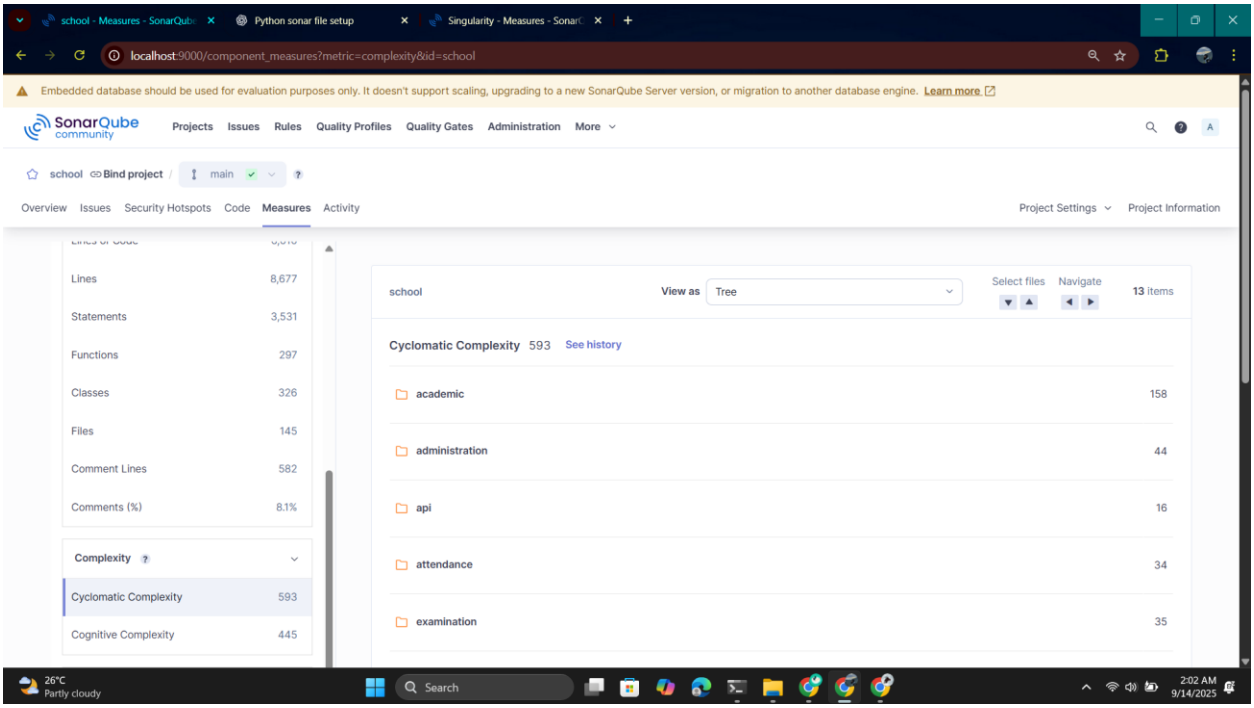
school = 4

sis = 49

templates = 1

users = 103

manage.py = 2



2. Cognitive Complexity

The total cognitive complexity of 445 indicates significant mental processing load required to understand the codebase, suggesting many functions likely exceed the recommended threshold of 15. This accumulated complexity poses challenges for developer comprehension and code maintainability across the system.

Overall cognitive complexity= 445

Here is the root folder cognitive complexity:

academic = 115

administration = 28

api = 14

attendance = 20

examination = 22

finance = 80

notes = 11

schedule = 21

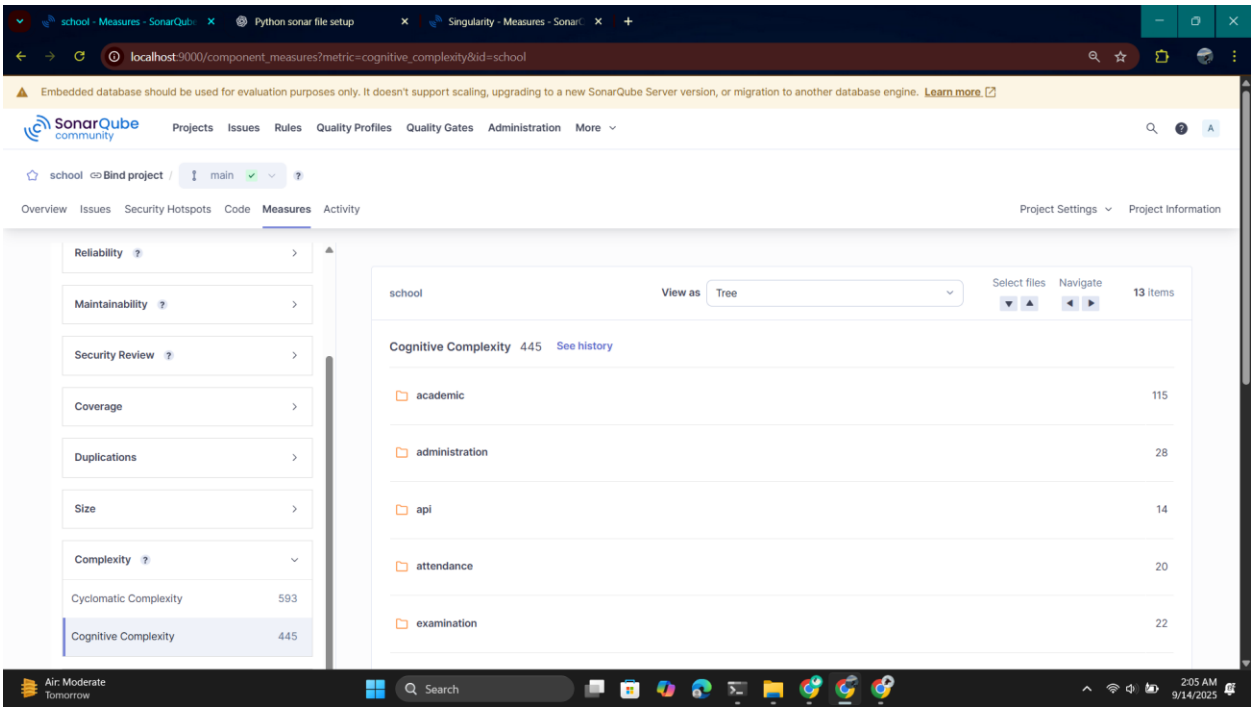
school = 0

sis = 58

templates = _

users = 74

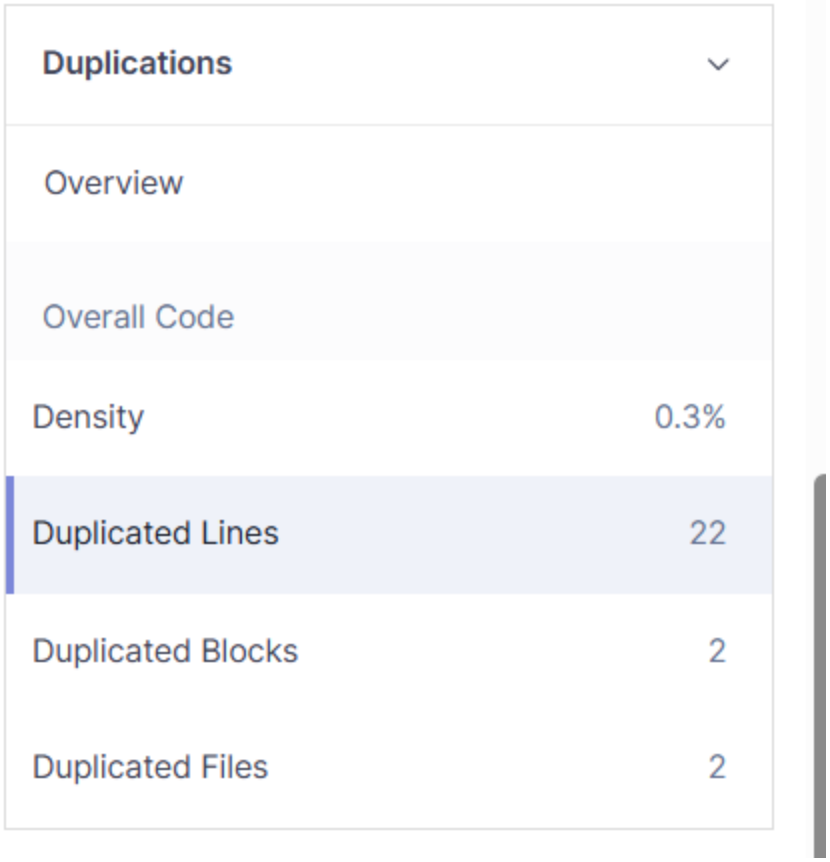
manage.py = 2

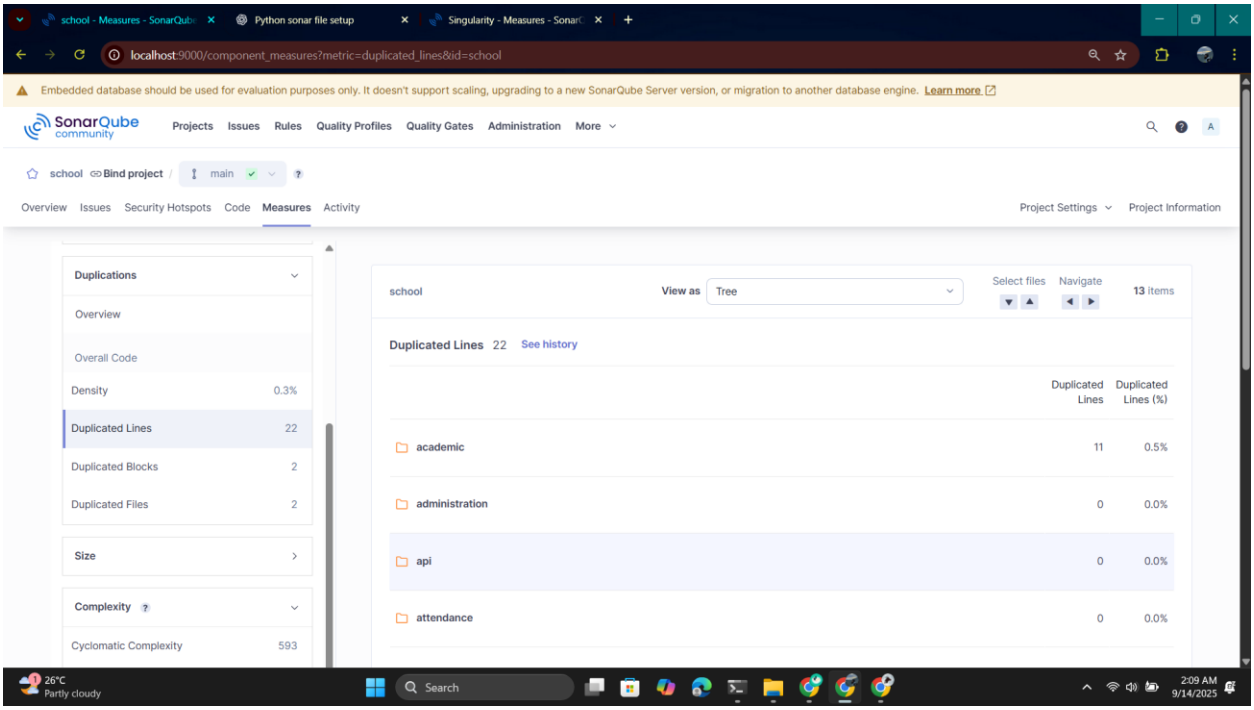


3. Code Duplication

Excellent duplication rate well below the recommended 3% threshold for code and significantly under the 5% overall target, indicating strong adherence to DRY (Don't Repeat Yourself) principles. This low duplication percentage demonstrates good code reusability and maintainability practices across the codebase.

Duplication density: 0.3%

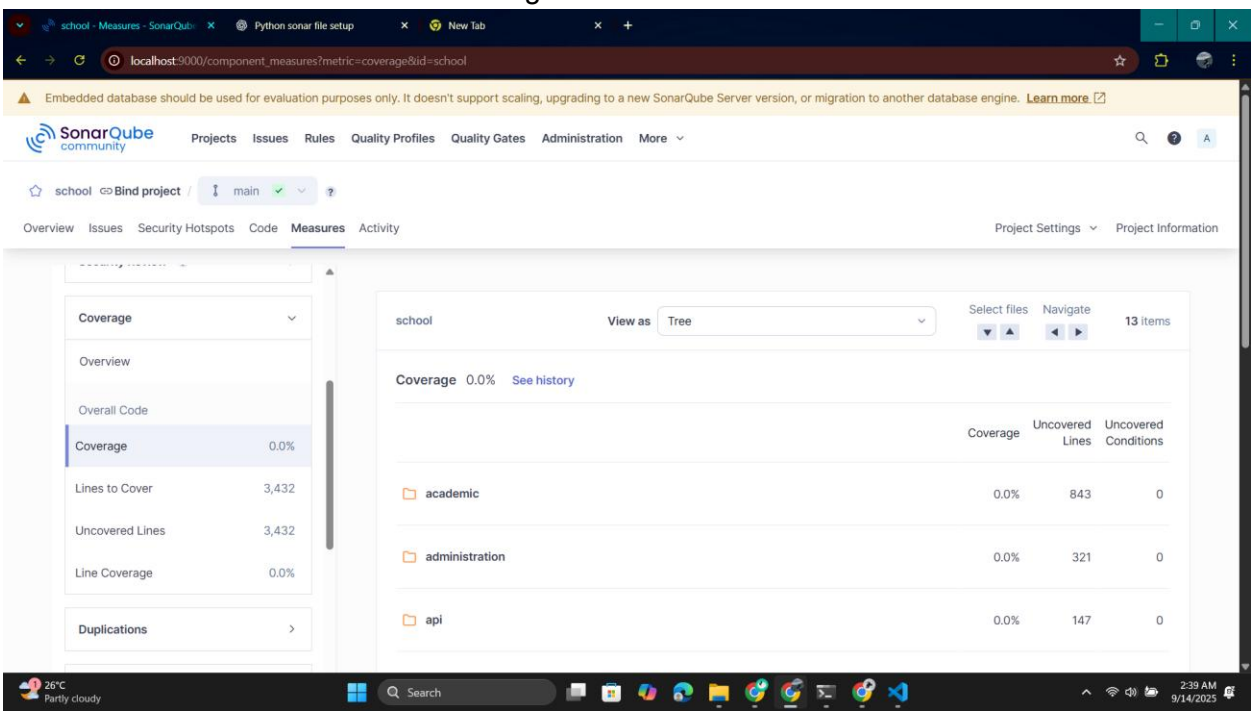




4. Test Coverage%

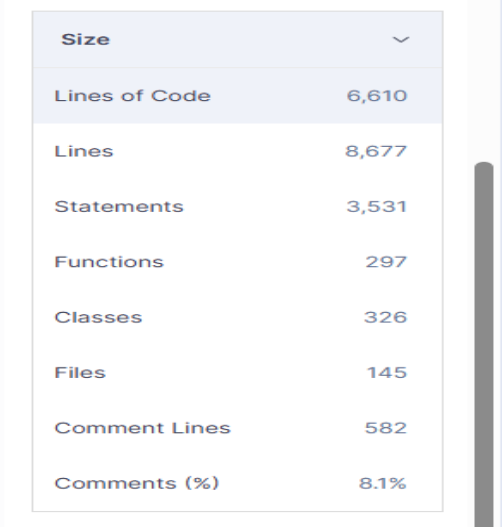
   Test coverage: 0.0%

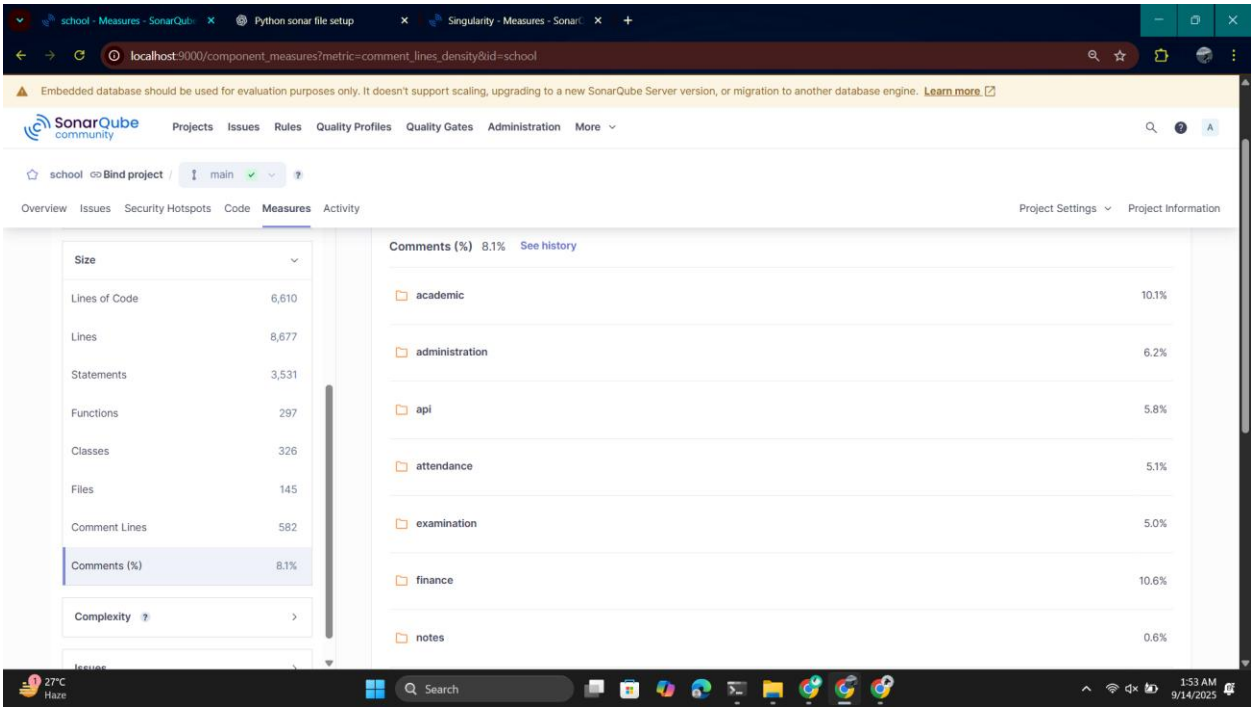
Critical deficiency with zero test coverage, far below the recommended 80% threshold, indicating complete absence of automated testing despite the presence of empty test files. This represents a significant technical debt and quality risk, as the entire codebase lacks verification through unit tests.



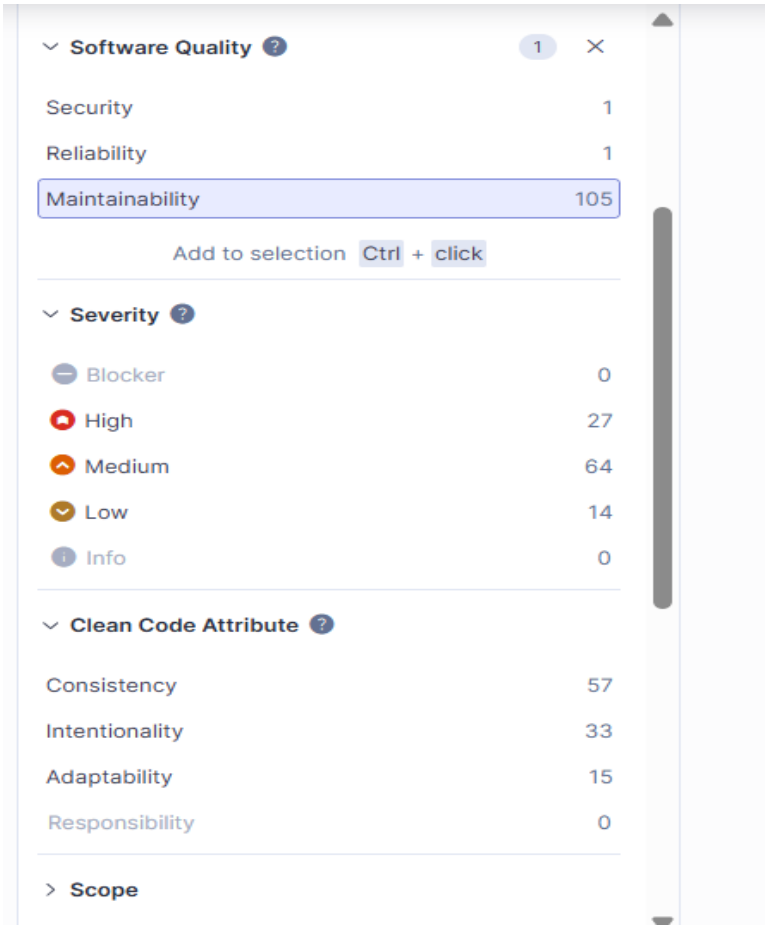
5. Comment density

Poor comment density at 8.1% falls significantly below the recommended 25% threshold, indicating insufficient code documentation that could hinder maintainability and knowledge transfer. The low comment ratio suggests developers may struggle to understand code purpose and functionality, creating potential technical debt in documentation practices.

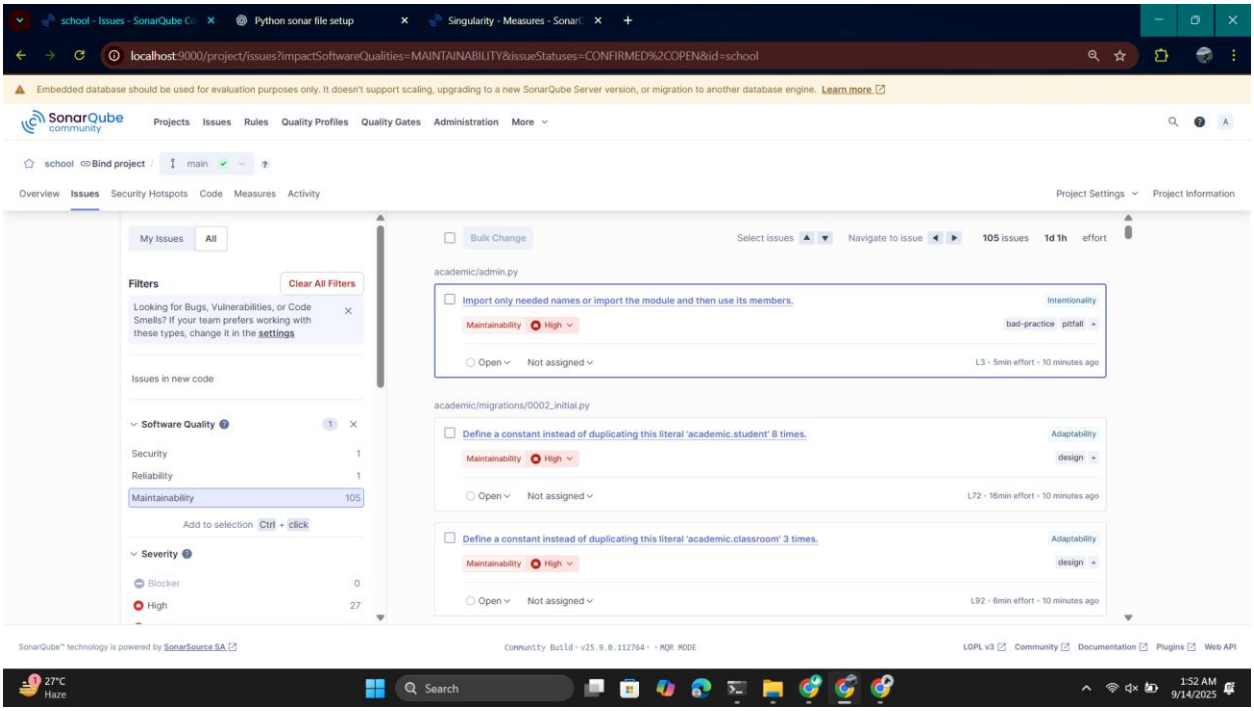




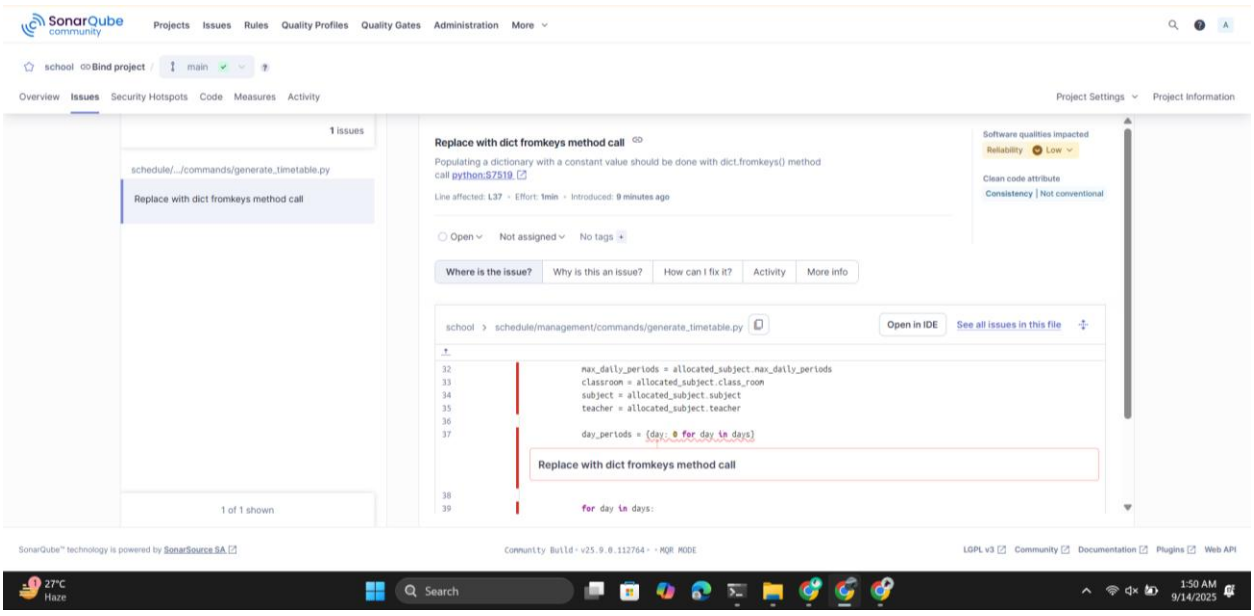
6. Bugs and vulnerabilities:



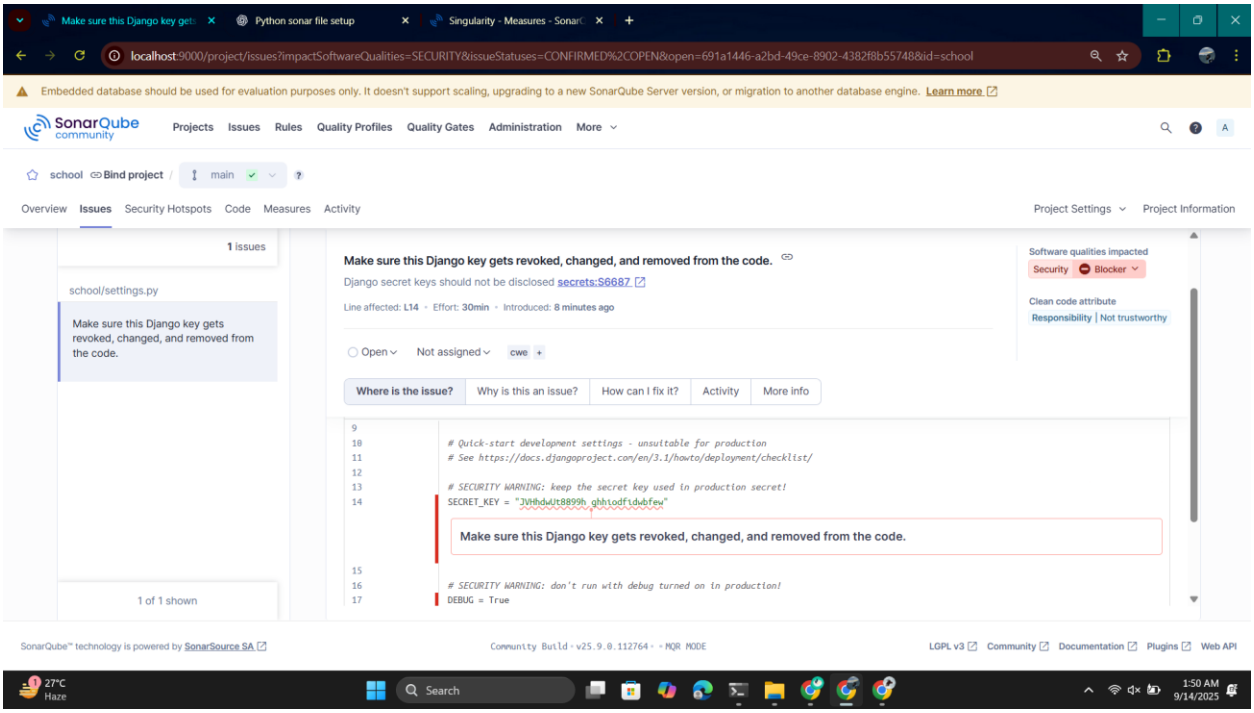
The 105 maintainability issues significantly exceed the zero new critical issues threshold, representing substantial technical debt in code smells, design problems, and coding standard violations. While these issues don't block functionality, they indicate a codebase that falls well short of the "A" quality rating standard and requires systematic remediation to achieve maintainable, clean code practices.



Low-severity reliability issue suggesting inefficient dictionary initialization using manual iteration instead of the more Pythonic dict.fromkeys() method. The code is creating a dictionary in a slow, complicated way when Python has a simpler built-in method to do the same thing faster.



Critical security blocker requiring immediate remediation - hardcoded Django secret key in settings.py violates security best practices and creates serious vulnerability. This exposed secret key compromises application security and must be moved to environment variables immediately to prevent potential unauthorized access.



o Dependency Mapping

1. Identify internal dependencies (module → module)

From the class diagram:

a. **Core Dependencies Flow:**

User (Django Auth) -> CustomUser/Admin -> Multiple View Classes -> SCMS_Models_Serializers -> Models

b. **Primarily Internal Dependencies:**

i. *CustomUser (Authentication Core)*

- Used by: Academic, Finance, Schedule, Admissions, Attendance, Exams
- Depends on: Django AbstractUser, Role-based permission system

ii. *SCMS_MODELS_SERIALIZERS (central serializers hub)*
   - All views across modules (Finance, Schedule, Academic, Attendance, Exams) rely on it
   - Depends on: all models from all apps

iii. *Academic module (Students, Parents, Teachers, Classes)*
   - Provides shared entities used in Finance, Attendance, Exams, Schedule.
   - Depends on: CustomUser.

iv. *Finance module*
   - Depends on: Academic (Students, Parents), CustomUser.
   - Used by: Views & Reports.

v. *Schedule module*
   - Depends on: Academic (Teachers, Subjects, Classes).
   - Used by: Attendance, Exams.

vi. *Exams module*
   - Depends on: Academic (Students, Subjects, Teachers), Schedule.

vii. *Attendance module*
   - Depends on: Academic (Students, Teachers, Classes), Schedule.

viii. ***Digital Notes / Materials module***
   - Depends on: Academic (Students, Teachers, Subjects).

ix. *Administration module*
   - Mostly standalone, but depends on: Academic (for academic years, terms).

2. Identify external dependencies (APIs, DBs, frameworks)

   Frameworks: Django, Django REST Framework.

   Database: Relational DB (Postgres/MySQL/SQLite) via Django ORM.

   Auth system: Django authentication + custom CustomUser.

3. Coupling Analysis

   CustomUser: Very central but acceptable (Auth is expected to be stable).

   SCMS_MODELS_SERIALIZERS: Too central; creates a monolithic bottleneck.

   Modules like Attendance, Exams, Finance: tightly coupled to Academic.

   Views: directly call serializers and models, no service layer → coupling risk.

4. Fan-in, Fan-out & Instability Metrics:

Component	Ca	Ce	I (0=stable, 1=unstable)
CustomUser	1	6	0.8 (unstable)
CustomUserAdmin:	3	0	1 (unstable)
CustomUserManager	0	1	1 (unstable)
DeltaRecord	0	2	1 (unstable)
Day	0	0	0 (stable)
CusotmExceptionMiddleware	0	0	0 (stable)
CustomUserChangeForm	0	1	1 (unstable)
CustomUserCreationForm	0	1	1 (unstable)
ClassYearSerializer	0	2	1(unstable)
ClassYearDetailView	0	1	1(unstable)

Concept	0	0	0 (stable)
ClassYearCreateView	0	1	1(unstable)
Meta	1	0	1(unstable)

3. Tool-Based Metrics (SonarQub)

- ## Cyclomatic Complexity

The total cyclomatic complexity of 593 across the entire codebase indicates significant accumulated complexity that requires systematic refactoring attention. While this represents the sum across all modules, individual functions should be evaluated against the ≤10 threshold for optimal maintainability.

Overall cyclomatic complexity = 593

Here is the root folder cyclomatic complexity:

academic = 158

administration = 44

api = 16

attendance = 34

examination = 35

finance = 110

notes = 22

schedule = 15

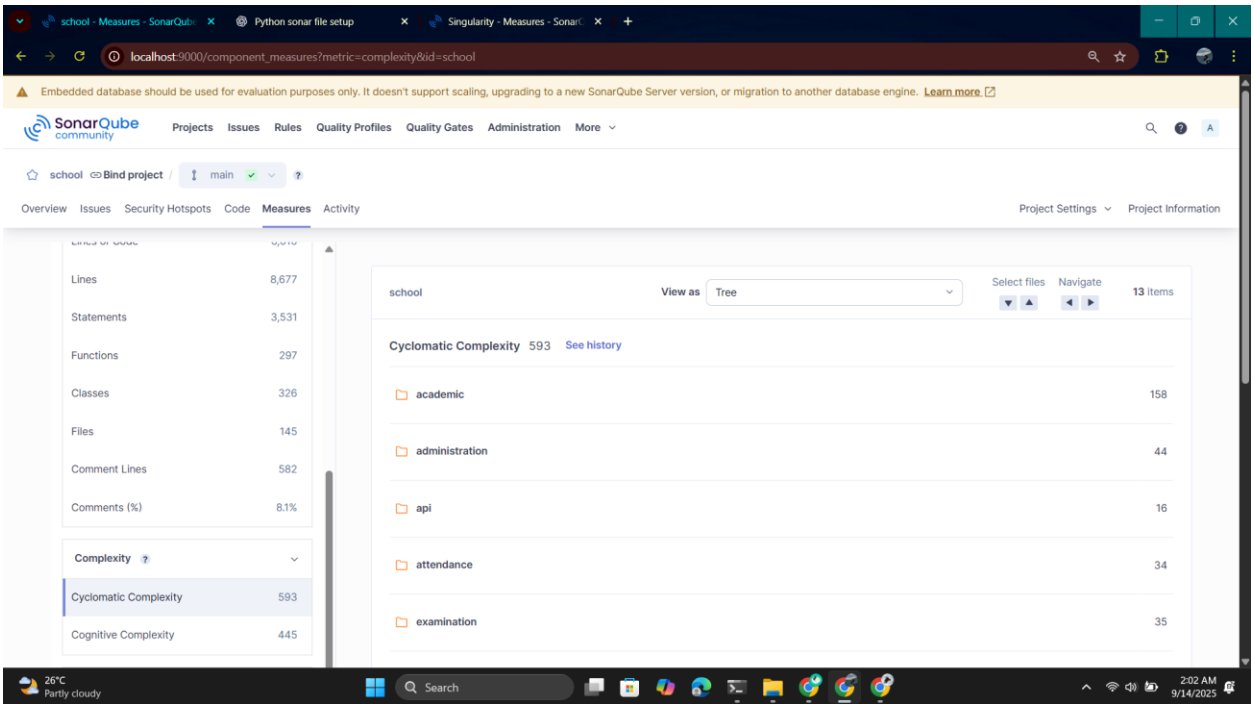
school = 4

sis = 49

templates = 1

users = 103

manage.py = 2



- ## Cognitive Complexity

The total cognitive complexity of 445 indicates significant mental processing load required to understand the codebase, suggesting many functions likely exceed the recommended threshold of 15. This accumulated complexity poses challenges for developer comprehension and code maintainability across the system.

Overall cognitive complexity= 445

Here is the root folder cognitive complexity:

academic = 115

administration = 28

api = 14

attendance = 20

examination = 22

finance = 80

notes = 11

schedule = 21

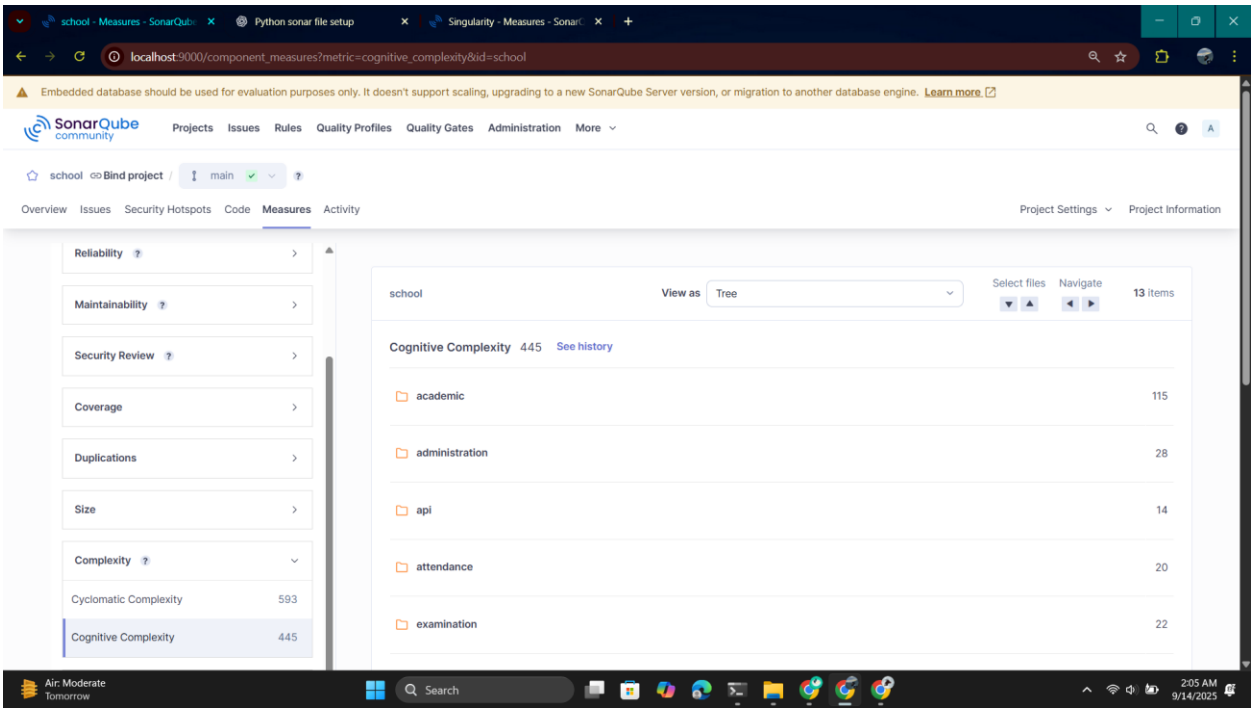
school = 0

sis = 58

templates = _

users = 74

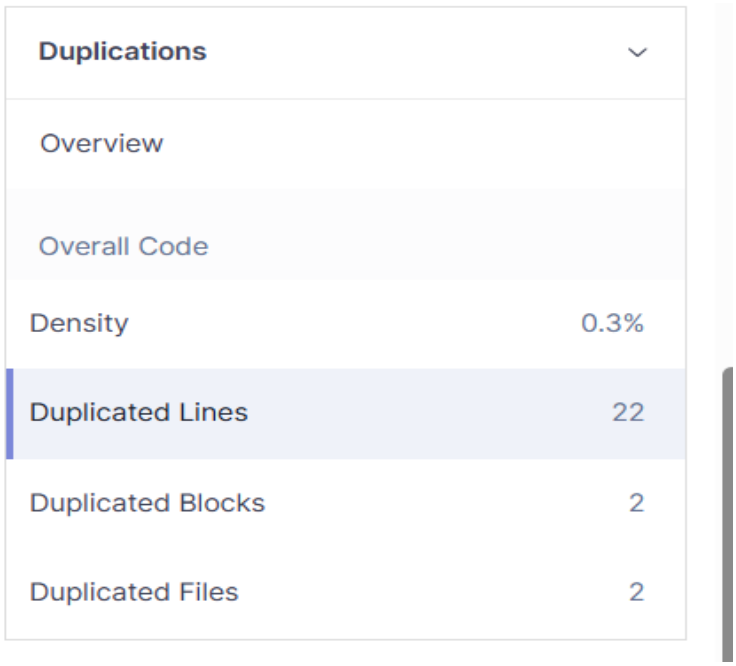
manage.py = 2

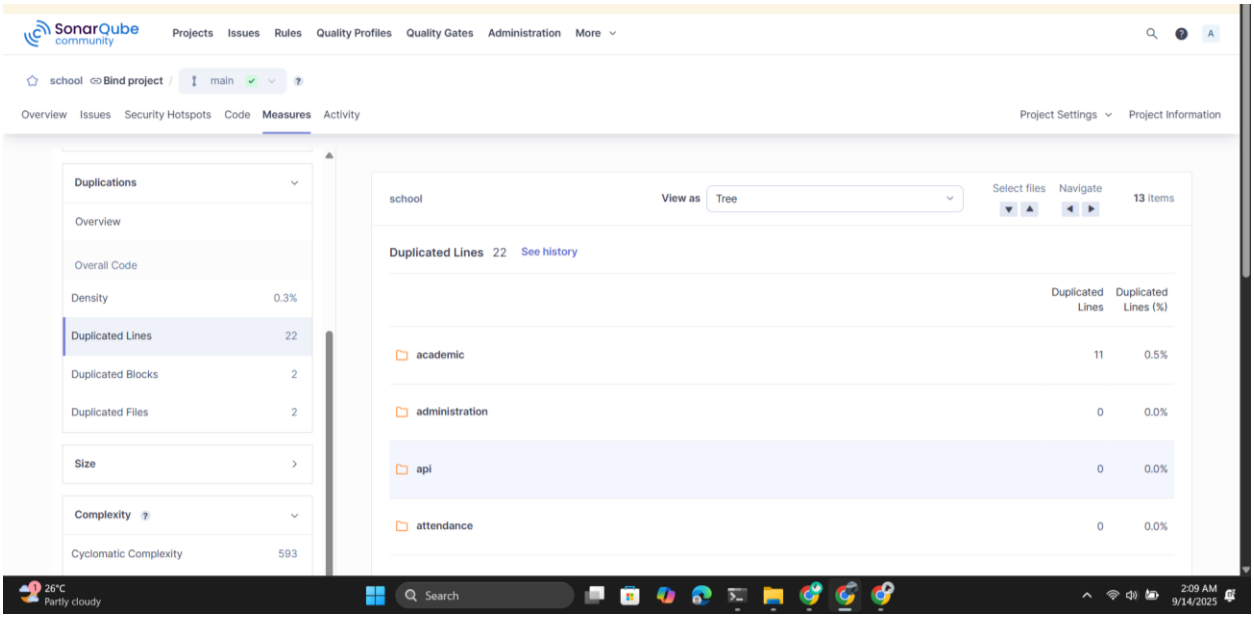


- **Code Duplication (%)**

Excellent duplication rate well below the recommended 3% threshold for code and significantly under the 5% overall target, indicating strong adherence to DRY (Don't Repeat Yourself) principles. This low duplication percentage demonstrates good code reusability and maintainability practices across the codebase.

Code Duplication = 0.3%



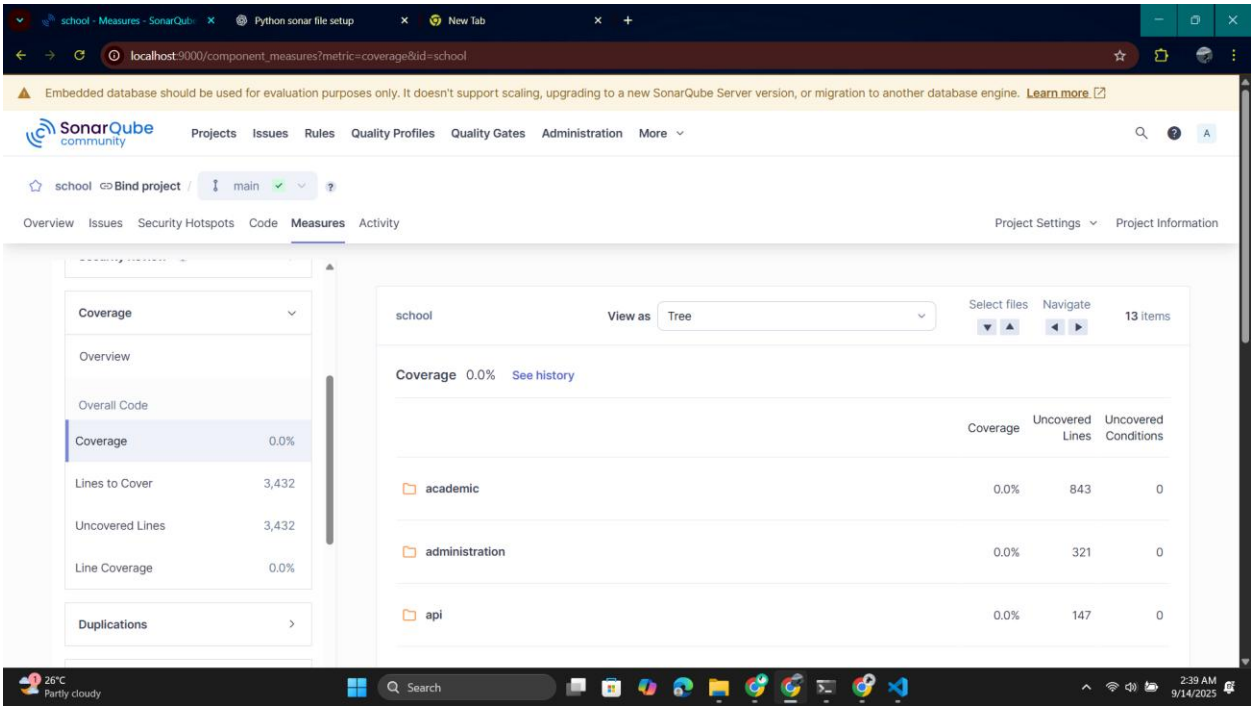


- **Test Coverage (%) (Statement and Branch)**

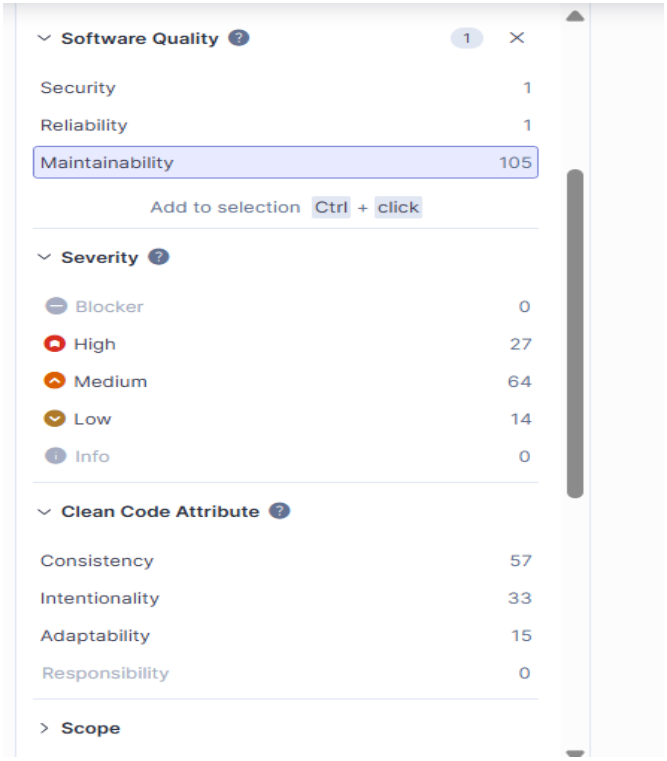
Critical deficiency with zero test coverage, far below the recommended 80% threshold, indicating complete absence of automated testing despite the presence of empty test files. This represents a significant technical debt and quality risk, as the entire codebase lacks verification through unit tests.

Overall Coverage = 0.0%

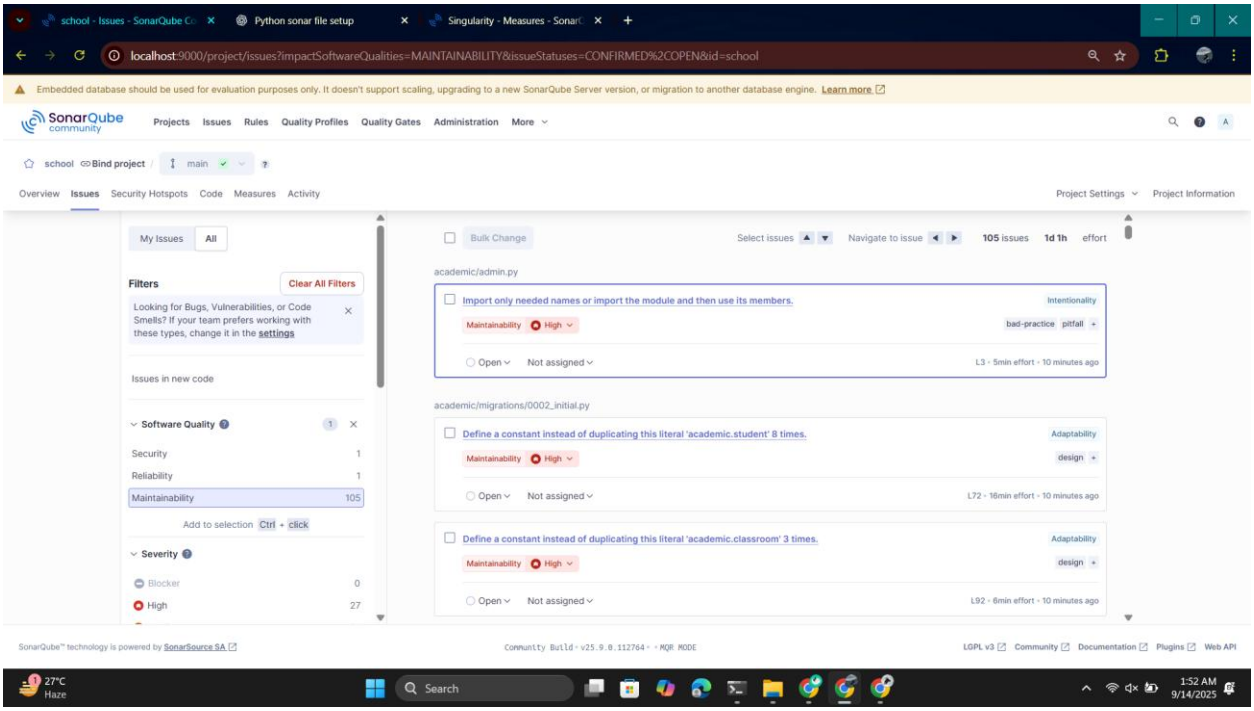
There are no files of test cases in this project, so no test coverage is possible.



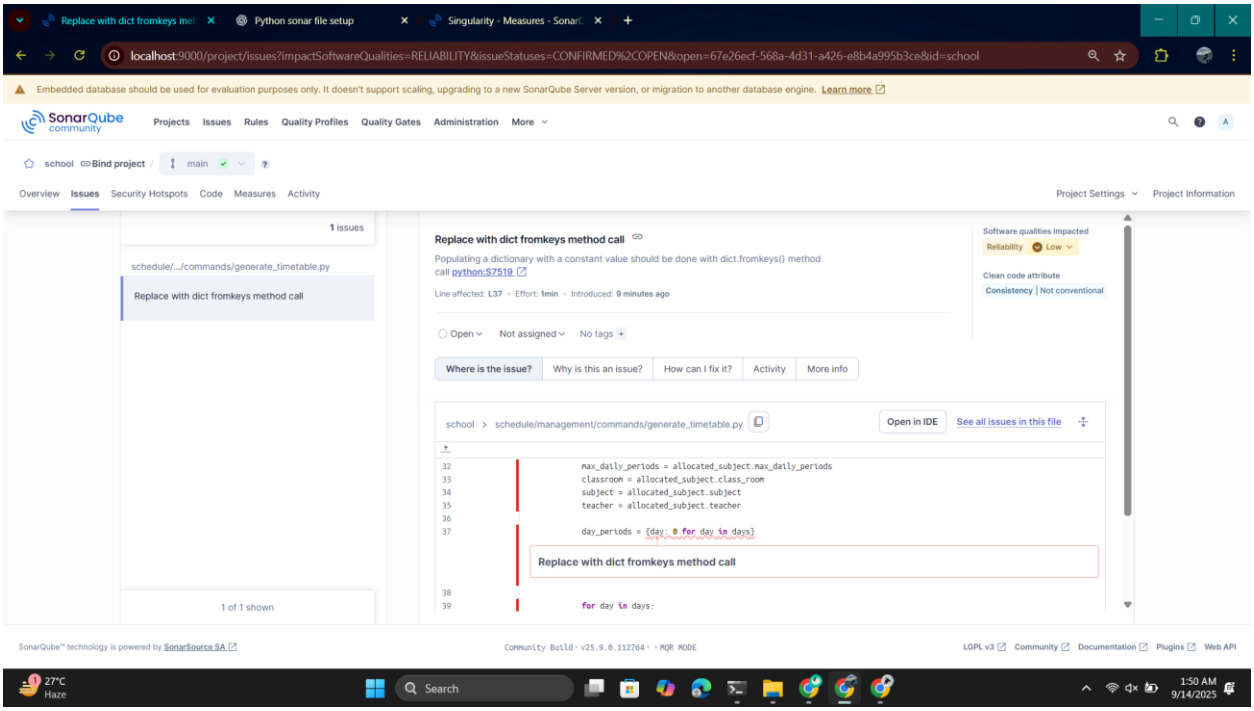
- **Bugs, Vulnerabilities, and Code Smells**



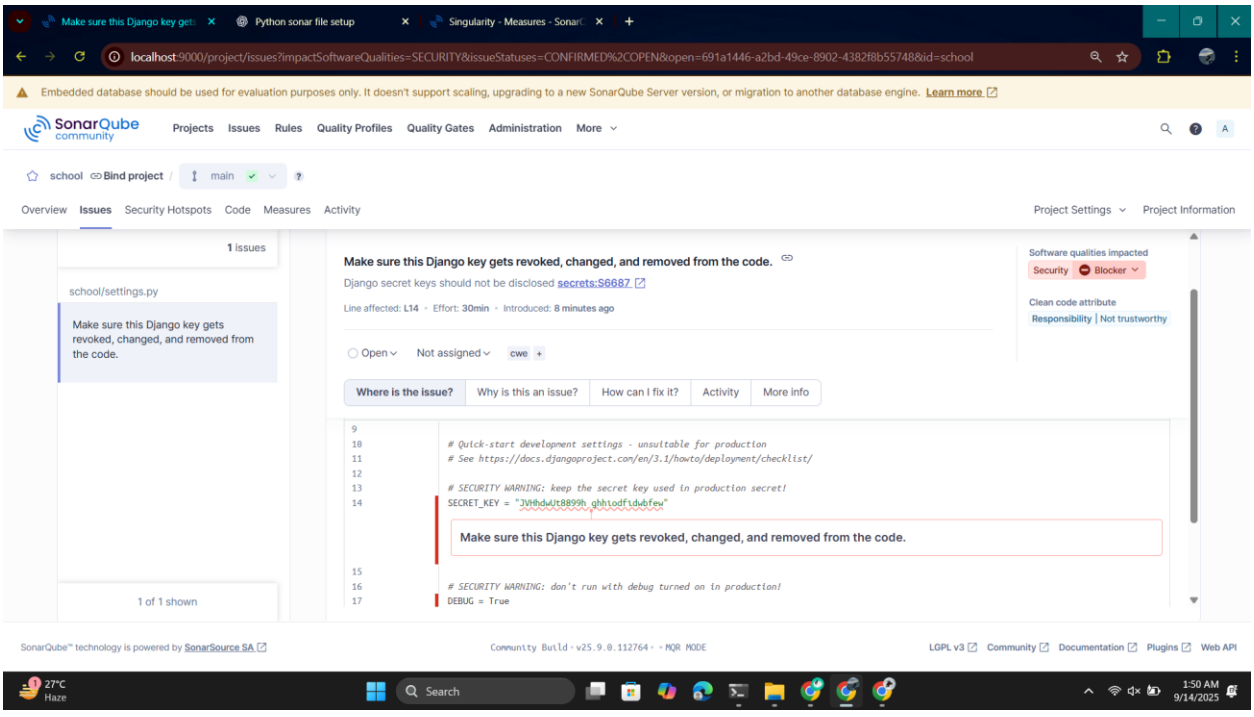
The 105 maintainability issues significantly exceed the zero new critical issues threshold, representing substantial technical debt in code smells, design problems, and coding standard violations. While these issues don't block functionality, they indicate a codebase that falls well short of the "A" quality rating standard and requires systematic remediation to achieve maintainable, clean code practices.



Low-severity reliability issue suggesting inefficient dictionary initialization using manual iteration instead of the more Pythonic dict.fromkeys() method. The code is creating a dictionary in a slow, complicated way when Python has a simpler built-in method to do the same thing faster.

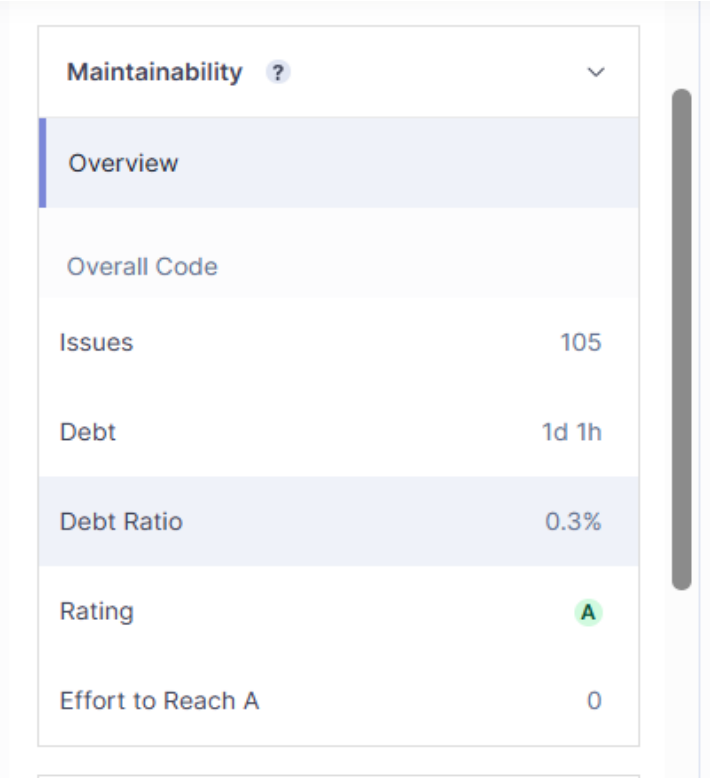


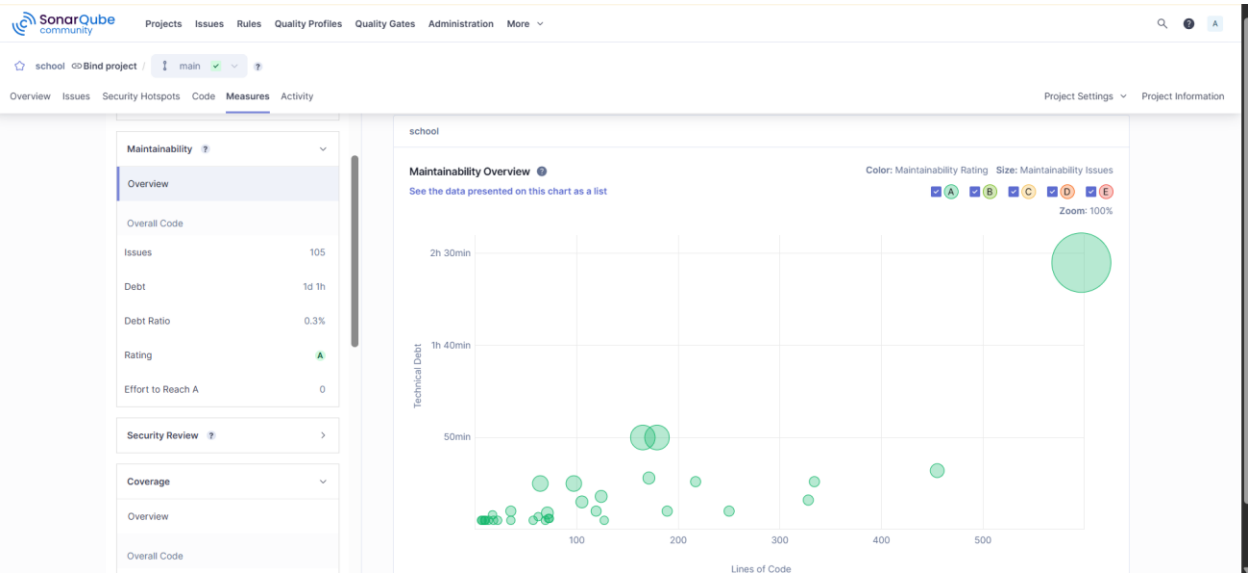
Critical security blocker requiring immediate remediation - hardcoded Django secret key in settings.py violates security best practices and creates serious vulnerability. This exposed secret key compromises application security and must be moved to environment variables immediately to prevent potential unauthorized access.



- **Maintainability Index**

Despite 105 maintainability issues requiring 1 day of remediation effort, the project achieves an excellent A-rating with only 0.3% debt ratio. This indicates that while code smells exist, the technical debt is minimal relative to the overall development investment and codebase size.





4. Debt Identification

Mapping of weaknesses to technical debt types

Three types of debt

1. Documentation Debt

   Weakness: Comments are not sufficient in the code. It is more difficult for new developers to comprehend and expand the system when there is missing, insufficient, or out-of-date documentation.

   Example: Only 35 comments are included in the 837 lines of code in the models.py file for our project, yielding a comment density of only 4.18% much less than the advised level. This low comment density suggests that the code lacks adequate documentation. Additionally, a few of the file's methods execute numerous jobs, which makes them intricate and challenging to understand. To improve readability, maintainability, and ease of future revisions, such procedures especially need clear and descriptive comments.

2. Test Debts

   Weakness: Missing or insufficient test cases lower system reliability and raise the possibility of undetected flaws. Every modification or bug repair must be manually confirmed in the absence of tests, which adds work, slows down development, and increases the possibility of introducing undetected flaws.

   Example: Every module and method currently lacks test cases.

3. Defect Debt:

   Weakness: The system still has known but unfixed defects, each of which is a liability that will eventually need work to address.

   Example: In config/settings/local.py, SonarQube highlights that the module is importing unnecessary components (Import only needed names or import the module and then use its members).

5. Remediation Cost and Debt Ratio

- Report tool outputs and explain the remediation cost calculation

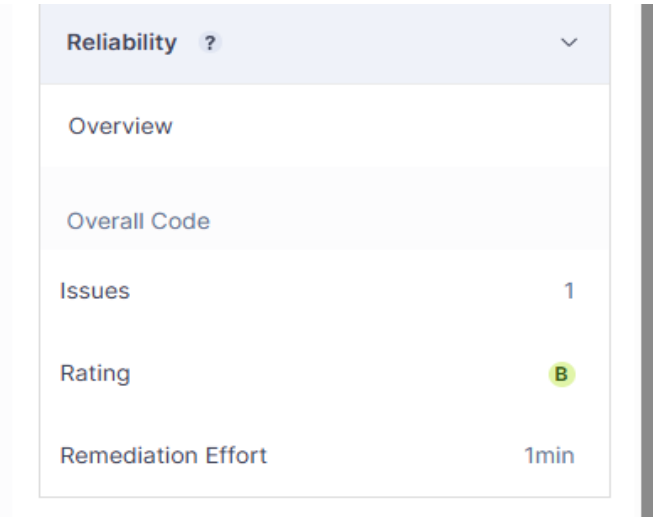
Total Remediation Cost Calculation according to sonarqube:

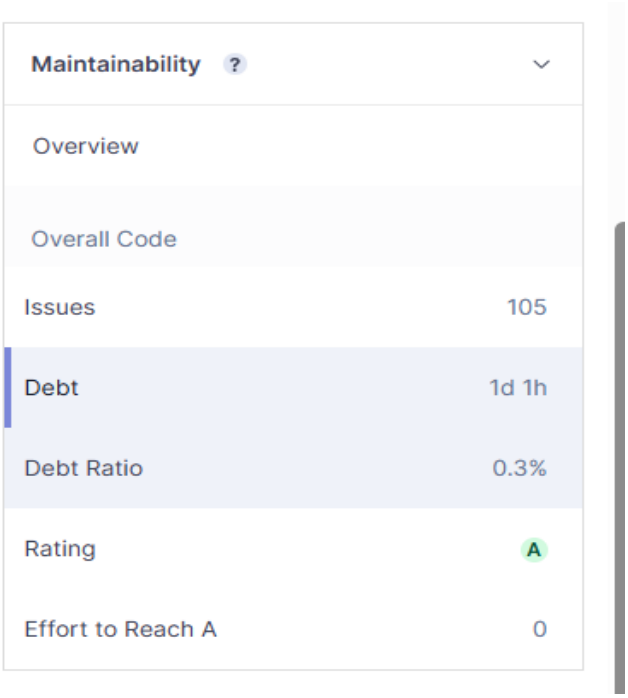
- Maintainability Debt: 1 day 1 hour (1d 1h)
- Security Issues: 1 blocker issue (30 min)
- Reliability Issues: 1 issue (1 min)

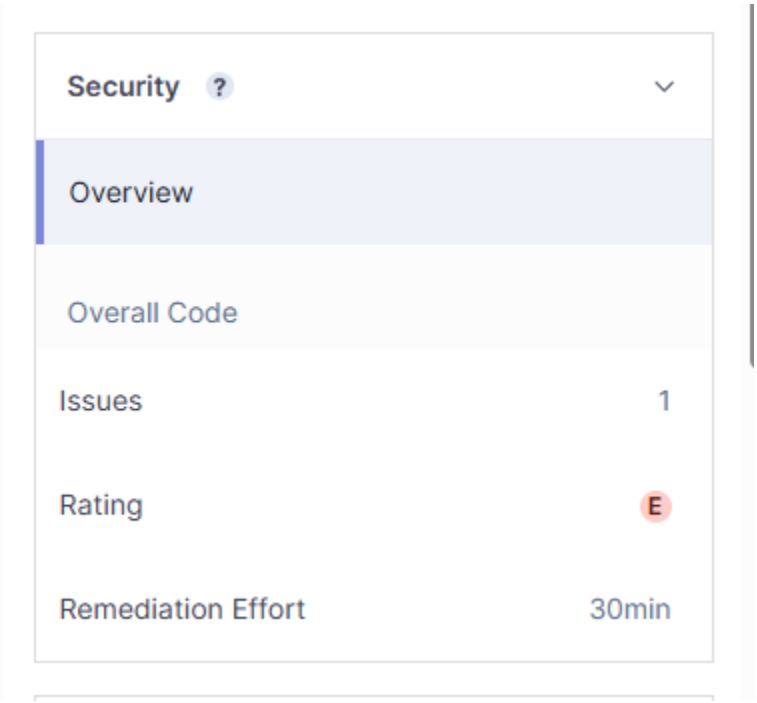
  Total: 1 day 1 hour 31 min

Total Estimated Remediation Cost: ~1 day 2 hours

Here are the detailed screenshots of the remediation costs:







- Compute the Technical Debt Ratio with justification

Given Data:

- Total LOC: 6,610
- Development Rate: 0.06 days/LOC
- Remediation Cost: 1 day 1 hour 31 minutes

Step 1: Calculate Total Development Cost

Total Development Time = LOC × Development Rate

Total Development Time = 6,610 × 0.06 = 396.6 days

Step 2: Convert Remediation Cost to Days

Remediation Cost = 1 day + 1 hour + 31 minutes

= 1 day + 1.517 hours

= 1 + (1.517 ÷ 24) days

= 1.063 days

Step 3: Apply Technical Debt Ratio Formula

Technical Debt Ratio = (Remediation Cost ÷ Total Development Time) × 100

Technical Debt Ratio = (1.063 ÷ 396.6) × 100

Technical Debt Ratio = 0.268% ≈ 0.27%

Justification: The calculated 0.27% debt ratio closely matches SonarQube's reported 0.3%, with the small variance likely due to:

1. SonarQube may use slightly different time estimates for issue remediation
2. The 0.06 days/LOC rate represents an industry average that may vary from actual project development patterns
3. Both calculations fall within the same quality tier

6. Prioritization Strategy

1. Risk-Based Prioritization:

Idea: Fix the debt that poses the highest risk to business value first.

Assessment:

- **Test Debt:** Any modification or feature addition may introduce undiscovered flaws into crucial modules (such as the booking and payment procedures) because there are no test cases at all. Because users and financial transactions may be directly impacted by workflow disruptions, this poses a significant business risk.
- **Defect Debt:** In contrast to missing tests throughout the project, known flaws (such as duplicate search results, UI misalignment, and SonarQube-detected reliability concerns) might have a localized impact on user experience and reliability.
- **Debt in the documentation:** Onboarding and maintenance are slowed down by low comment density (4.18%) and missing guidance, but the immediate business risk is smaller than that of functional failures.

Prioritization Order (by risk):

Test Debt (highest risk to functionality & system reliability)

Defect Debt (moderate risk, affects quality but isolated issues)

Documentation Debt (low direct risk, long-term maintainability issue)

2. Cost of Delay Analysis:

Idea: Fix the debt that becomes more expensive over time first.

Assessment:

- Defect Debt: If left unfixed, problems (such as incorrect billing logic or reliability issues identified by SonarQube) amplify their impact. It is less expensive to fix now than to fix dependencies or corrupted data later.
- Test Debt: Since it becomes more difficult and costly to retrofit tests for a larger codebase later on, the cost of not having tests rises with each new feature.
- Debt from documentation: The cost increases more slowly than other types. Debugging a system without testing is more costly than updating documentation later.

Prioritization Order (by cost of delay):

Defect Debt (fixing bugs early prevents cascading failures and costly rework)

Test Debt (test coverage is cheaper to implement earlier; retrofitting later is costly)

Documentation Debt (delaying has relatively low cost increase)

7. Reflection

Lessons Learned: We discovered how crucial it is to handle technical debt as a quantifiable and traceable component of software development while completing this task. We were able to comprehend the direct relationship between measures like cyclomatic complexity, duplication %, comment density, and test coverage and long-term maintainability thanks to tools like SonarQube. Additionally, we discovered that measures for fan-in, fan-out, and instability index, as well as dependency mapping, expose hidden hazards including over-coupled modules and central bottlenecks.

Trade-offs: We found trade-offs between long-term maintainability and short-term delivery speed during our investigation. For instance, the project entirely forgoes test coverage, introducing a high reliability risk, while achieving extremely low duplication, which is beneficial for maintainability. In a similar vein, employing a central serializer hub (SCMS_MODELS_SERIALIZERS) makes evolution more difficult by increasing coupling and instability while temporarily simplifying code reuse. These compromises demonstrated to us how software development teams frequently put immediate functionality ahead of architectural excellence.

Implications: The assignment made clear that neglecting technical debt now causes delays, but it increases maintenance costs later. Because there are no automated tests for this project, all future modifications will need to be manually validated, which will slow down

development. While connectivity across modules (e.g., Academic ↔ Finance ↔ Exams) increases regression risk when making simple changes, low documentation density also makes it difficult for new developers to get started.

In the future, maintainability would be greatly increased by lowering instability in crucial components, enhancing documentation, and increasing test coverage. By doing these actions, it will be possible to add new features as the project develops without having to continuously refactor the outdated source. To put it briefly, proactive debt management is necessary for sustainable evolution rather than its neglect.

B. Individual Component

A. Member 1 (Amna)

1.2 Files

Code File 1: schedule/management/commands/generate_timetable.py (62 LOC)

Code File 2: users/models.py (97 LOC)

2. Cyclometric Complexity and Cognitive Complexity

Cyclometic Complexity:- (1+# paths)

File #1:- generate_timetable.py:-

→ Class Command:-

A.) def handle(self, *args, **kwargs)
+1 (method)

B.) if(not current_term):
+1 (no nesting)/depth:0

C.) if(not allocated_subjects):
+1 (depth: 0)

D.) for (allocated_subject in allocated_subjects):
+1 (depth: 0)

E) for day in days:
+1, depth: 1 → (1+1) ← total

F.) for period_index in range( ):
+1, depth: 2, (total: 1+2)

G.) if period_index == 4:
+1, depth: 3, (total: 1+3)

H.) if(day_period >= weekly_Limit
or
consecutive_periods >= max_periods)
+1, depth: 3, total (1+3)
(or: doesn't add 1)

Total CC = 1+1+1+1+2+3+4+4(+1)
" = 17 + 1 → [redacted]

File # 2:- users > model.py:-

→ Class CustomUser :-

A.) def __str__(self) → +1

→ Class Accountant :-

B.) def __str__self → +1

C.) def deleted(self): → +1

D.) def save(self, *args, **kwargs) → +1

E.) if (not self.username)
+1 , depth : 0

F.) if (not self.user)
+1 , depth : 0

G.) if (self.empId & len(self.empId)
+1 , depth : 0

H.) def update_unpaid_salary(self) → +1

I.) if self.unpaid_salary > 0
+1 , depth : 0

Total CC = 1+1+1+1+1+1+1+1+1+1

Total CC = 9.

Cognitive Complexity:-

File 1:- timetable.py:-

→ class Command:

A.) def handle(self, *args, **kwargs) — +0

B.) if not current_term : → +1 , depth : 0

C.) if not allocated_subjects: → +1 , depth : 0

D.) for allocated_subject in allocated_subjects → +1

E.) for day in days: → +1 , depth : 1 (1+1)

F.) for period_index in range(): → +1, depth : 2

G.) if period_index == 4:
+1 , depth : 3 (1+3)

H.) Continue ⟶ +1 , depth: 4 (1+4)

I.) if (period_day >= WeeklyLimit or consecutive_periods >= max_period)
+1 , depth: 3 (1+3)

J.) break ⟶ +1 , depth: 4 (1+4)

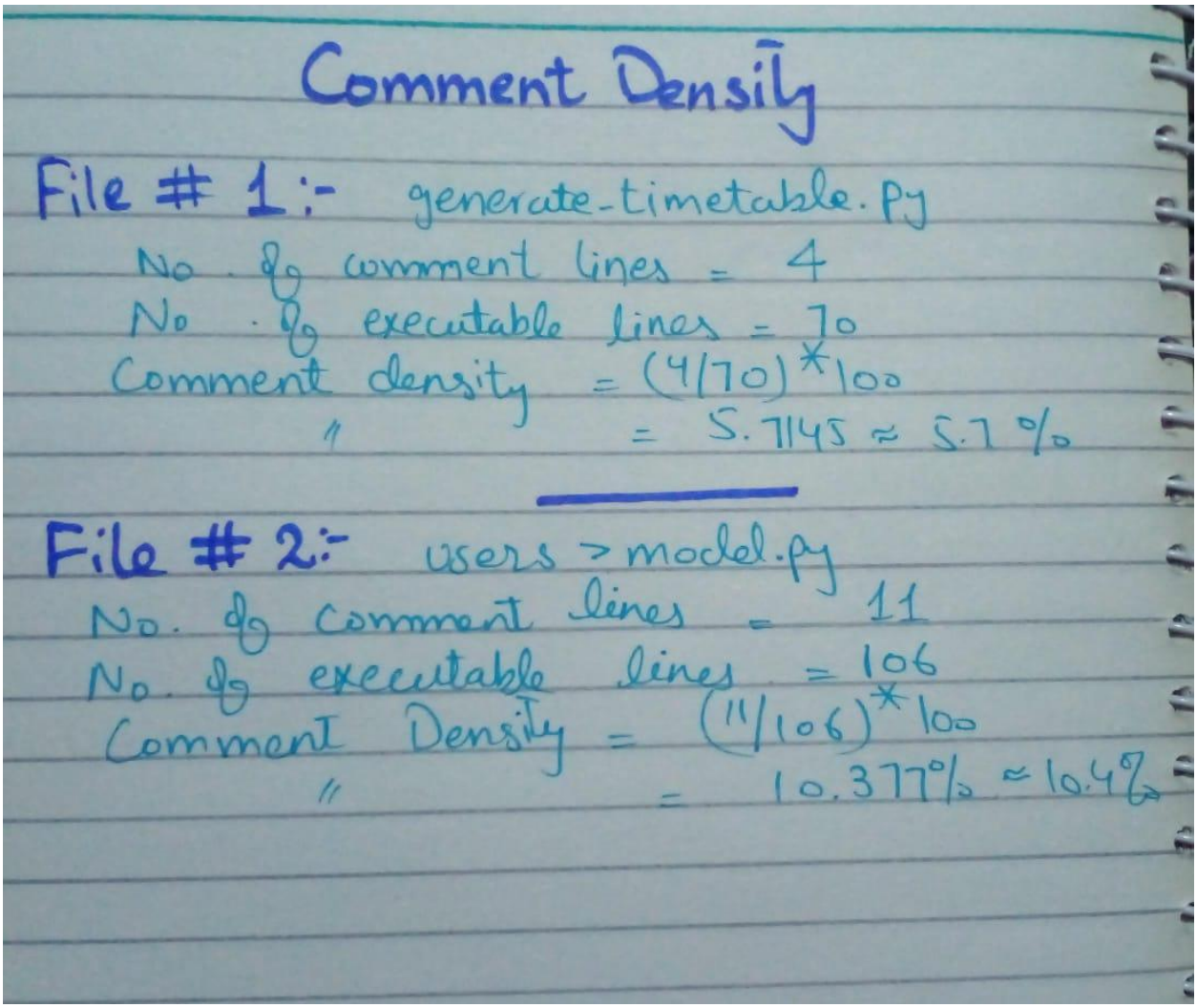
Total CoC = 1+1+1+2+3+4+5+4+5
" = ~~37~~ 26

File#2:- Users > models.py:-

A.) def save(self): ⟶ 0
B.) if (not self.username):
+1 , depth: 0
C.) if (not self.user):
+1 , depth: 0
D.) if (not self.empId && len(empId)
+1 , depth: 0)
E.) def update_unpaid_salary(self) ⟶ 0
F.) if self.unpaid_salary > 0
+1 , depth: 0

Total CoC = 1+1+1+1 = 4

3.Comment Density



4. **Short Interpretation**

File 1:

This file is extremely complicated, surpassing the suggested thresholds (≤10 for cyclomatic and ≤15 for cognitive), as seen by its cyclomatic complexity of 17 and cognitive complexity of 26. This suggests that it will take a lot of mental work to comprehend and alter the code, increasing the possibility that mistakes will be introduced during the process. This problem is made worse by the low comment density (5.7%), which leaves future developers with less reference to go to when browsing code.

This file adds to design debt and maintainability debt from the standpoint of technical debt. Excessive complexity indicates that refactoring is necessary, such as dividing logic into smaller functions or using design patterns to make the code easier to comprehend. Without these

precautions, files run the risk of becoming unmaintainable, which will eventually make bug repairs and feature updates more expensive.

File 2:

The code in this file is comparatively simple and easy to understand, with a cyclomatic complexity of 9 and a cognitive complexity of 4, both of which fall below suggested criteria. There is not enough inline documentation, though, as the comment density is only 10.3%, which is still less than the suggested 25%. Even though the code is now manageable, the absence of comments may cause issues as the system develops or more developers join the project.

This file is more documentation debt than design debt in terms of technical debt. The immediate risk is modest due to the reasonable code complexity, but insufficient comments may eventually cause debugging, onboarding, and maintenance tasks to lag.

B. Member 2 (Kashaf)

1.Files

File 1: academic/serializers.py (78 LOC)

File 2: api/middleware.py (57 LOC)

2. Cyclometric Complexity and Cognitive Complexity

&

3. Comment Density

Kashaf Fatima

Day: 22i-2415 Date: __/__/20

① academic/serializers.py

```
class SubjectSerializer(serializers.ModelSerializer)
    def validate_subject_code(self, value):          → +1
        if (len(value) < 3:          → +1          → +1

Class ClassRoomSerializer(serializer.ModelSerializer):
    def get_name(self, obj):          → +1

    def get_stream(self, obj):          → +1

    def get_class_teacher(self, obj):          → +1
return (if obj.class_teacher)          → +1          → +1

Class StudentClassEnrollmentSerializer(seri...)
    def validate(self, data):          → +1
        if classroom.occupied_sits >= classroom.capacity:
                ↳ +1          ↳ +1
```

Cyclomatic Total = 8

Cognitive = 3

Comment density:

Total LOC = 78

comments = 3

comment density = $\frac{3}{78} \times 100 = 3.84\%$

Day: _____ Date: __/__/20

② api/middleware.py

```
class CustomExceptionMiddleware:
    def __init__(self, get_response):                    → 1

    def __call__(self, request):                         → 1
        except ValidationError as e:                     → 1   -1
            status = 400

        except                                           → 1   -1
            if (hasattr)                                 → 1   -2
            status = 404

        except                                           → 1   -1
            status = 403

        except:                                          → 1   -1
            status = 400

        except=                                          → 1   -1
            status = 500

        except=                                          → 1   -1
            status = status_code

        except:                                          → 1   -1
            status = 500
```

Cyclomatic: 10 Cognitive = 9	Commend density: $\frac{3}{57} \times 100$ = 5.26%

4. **Short Interpretation**

File 1:

This file has a cyclomatic complexity of 8 and cognitive complexity of 3, both within safe thresholds, so it is relatively easy to follow. The main risk lies in the very low comment density (3.84%), which limits context for future maintainers. Some serializer methods assume related objects are always present, which may raise errors if null values appear. The enrollment validation also risks race conditions if multiple requests try to enroll students simultaneously.

From a technical debt perspective, the repeated use of fields = "__all__" tightly couples serializers to models, making changes harder to manage. Domain logic (e.g., subject code and class capacity checks) embedded in serializers instead of models or services increases maintenance overhead. Improving documentation, handling null checks defensively, and moving validation rules closer to the data layer would help reduce long-term debt.

File 2:
Complexity is moderate (cyclomatic 10, cognitive 9) and the flow is linear, but the low comment density (~5.26%) leaves intent undocumented. Risks include broad exception swallowing: converting all errors to JsonResponse can hide bugs and leak internal messages (e.g., raw ValidationError/APIException details). Only the catch-all path is logged; specific errors aren't, which may hinder incident triage. Middleware ordering and DRF vs. Django exception handling could also conflict with existing global handlers.

From a debt/maintenance view, the many near-duplicate except blocks suggest a mapping table or helper to reduce repetition and keep status/message formats consistent. Hard-coding JSON responses bypasses DRF's standard exception_handler, limiting content negotiation and reuse. Consider: a dict-based dispatcher, structured error codes, consistent logging for all branches, environment-aware detail (hide messages in prod), brief docstrings, and unit tests asserting status codes and payload shapes.

C. Member 3 (Safi)

1. Files

File 1: notes/serializers.py (71 LOC)

File 2: administration/serializers.py (69 LOC)

2. Cyclometric Complexity and Cognitive Complexity

&

3. Comment Density

DATE / /

RE Engg:-

assignment:-

notes/serialize.py

class stringSerializer(serializers.StringRelatedField)

def to-internal-value (self, value)

↘ (+1)

Class Assign~~ment~~ materializer

def get-questions (self, obj):

↘ (+1)

def create (self, request) → (+1)

for q in data ["questions"] → +1

↘ (+1)

for c in q ["choice"]

↘ (+1) ↳ +2

Class GradeAssignment Serializer

~~def~~ def create (self, request) - (+1)

questions = [q for q in ...] → +1 (+1)

answer = [data ["answer"] [a] for a in ...] (+1)

↘ +1

DATE / /

S M T W T F S
O O O O O O O

for i in range(len(questions)) → (+1)

if questions[i].answer.title = answers[i]

→ +2 (+1)

Cyclomatic = 10 → circled.

Cognitive - 8

DATE / / S M T W T F S

administration / serializer.py

Class ArticlesSerializer

def get_created_by(self,obj) → (+1)

if serializer.data["first_name"] → (+1)

↳ +1

def get_short_content(self,obj) → (+1)

Cyclomatic = 3

Cognitive = 1

Comment densities :-

notes / serializer.py

(0/71) x100 = 0 %.

administration / serializer.py

(1/69) x100 = 1.4 %.

4. **Short Interpretation**

File 1:

Complexity is moderate (cyclomatic 10, cognitive 8), but risk is elevated by business logic living inside serializers. The create(self, request) signatures bypass DRF's normal create(self, validated_data) path and pull from request.data, skipping validation and making tests harder. Direct lookups like User.objects.get(username=...), Assignment.objects.get(id=...), and Choice.objects.get(title=...) can raise DoesNotExist; using title for answers assumes uniqueness. The grading logic relies on positional alignment between questions and answers, which is brittle, and there's a stray print plus no atomic transaction around multi-row writes.

From a debt/maintenance view, serializers are mixing persistence, orchestration, and grading rules, which will be costly to change. Consider moving creation/grading to services, switching to validated_data with explicit field lists (avoid __all__), adding uniqueness/foreign-key validations, and wrapping creates in transaction.atomic(). Prefer IDs over titles for answer selection, guard against missing keys, and replace get_questions/nested access with prefetching in views to curb N+1s. Adding brief docstrings and tests for the create/grade paths will stabilize future maintenance.

File 2:

With cyclomatic 3 and cognitive 1, this file is simple to follow, but the very low comment density (~1.4%) is a risk for future changes. ArticleSerializer.get_created_by serializes the user per row and then digs into serializer.data, which is wasteful and can trigger N+1 queries unless views use select_related. get_short_content naively slices to 200 chars, which may cut words/HTML. In SchoolEventSerializer, the field named academic_year exposes a string via source="term.academic_year.name"—that name collides with the likely FK concept and can confuse API consumers. There's also no validation for date ranges (start_date < end_date) on Term/SchoolEvent.

From a technical-debt and maintenance perspective, mixing derived display fields with identifiers without clear naming (e.g., academic_year_name) increases coupling and migration friction. Consider preloading relations in views, returning obj.created_by.first_name or obj.created_by.email directly, renaming display fields (academic_year_name) consistently, adding brief docstrings, and moving cross-model rules (like date ordering) to model validators. Avoid __all__ where used, and prefer explicit field lists to prevent accidental API surface changes.

Bibliography

[1] Cyclomatic complexity and Cognitive complexity threshold:
https://devguide.trimble.com/development-practices/managing-code-complexity/#:~:text=,depending%20on%20the%20application%20domain

[2] Code Duplication and Test Coverage Threshold:
https://docs.sonarsource.com/sonarqube-server/10.8/instance-administration/analysis-functions/quality-gates/#:~:text=,0

[3] Maintainability Index:
https://docs.sonarsource.com/sonarqube-server/8.9/user-guide/metric-definitions/?utm_source=chatgpt.com

[4] Comment density:
https://mdtproductdevelopement-enablement-pnps-dev.azurewebsites.net/Tools/SonarQube/Best-Practices/SonarQube-Standards-and-Best-Practices/?utm_source=chatgpt.com